GlucoTrack: Documento di Requisiti e Interazioni Utente-Sistema

---

1. Introduzione al Sistema

GlucoTrack è un’applicazione desktop sviluppata in **Java**, progettata per supportare pazienti, medici e amministratori nella gestione e nel monitoraggio della glicemia e delle terapie farmacologiche. Il sistema offre un ambiente sicuro e intuitivo per la registrazione delle misurazioni, la gestione dei sintomi, la prescrizione dei farmaci e la consultazione dei dati clinici. Attraverso dashboard personalizzate, GlucoTrack facilita la comunicazione tra i diversi attori e promuove una gestione efficace della salute, garantendo al tempo stesso la sicurezza e la riservatezza dei dati sensibili. La modularità e l’adozione di best practice di sviluppo rendono l’applicazione facilmente estendibile e adattabile a future esigenze.

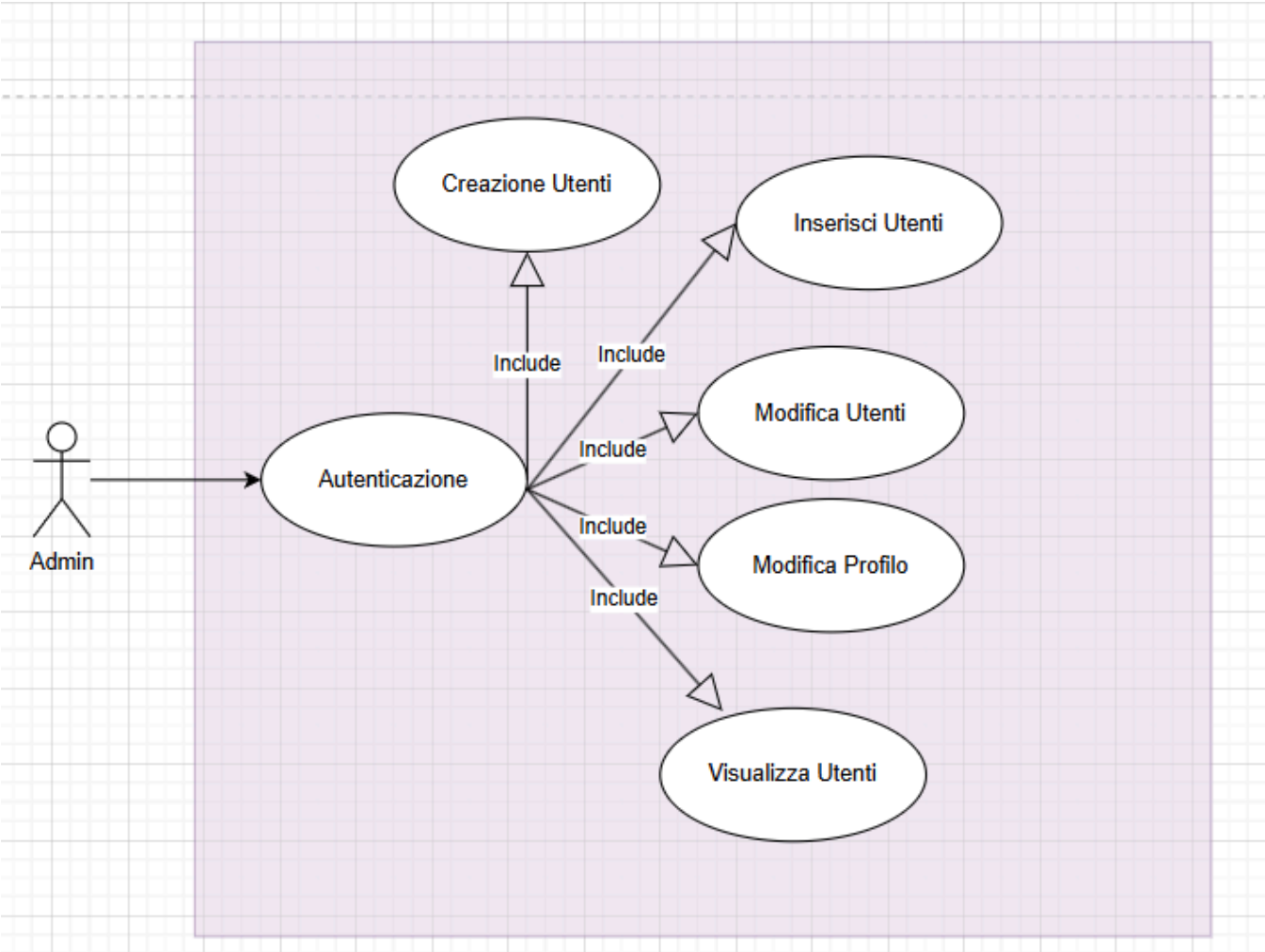
---

2. Requisiti e Interazioni Utente-Sistema

Il sistema GlucoTrack prevede la registrazione, consultazione e gestione di dati clinici (misurazioni glicemiche, sintomi, terapie) da parte di diversi utenti. Un utente può essere paziente, medico o amministratore, ciascuno con funzionalità e permessi specifici.

2-1. Casi d'uso per l'Amministratore (Admin)

Dopo aver effettuato l’autenticazione, all’utente Admin vengono rese disponibili tutte le azioni necessarie per la gestione degli utenti.



2.1.1 Creazione di un nuovo utente

- **Attore:** Admin
- **Precondizioni:** Utente Admin autenticato
- **Passi:**
  i. L'admin accede al sistema
  ii. L'admin seleziona "Nuovo utente"
  iii. L'admin inserisce i dati richiesti tramite l'interfaccia dedicata
  iv. Il sistema verifica la validità e unicità dei dati
- **Postcondizioni:**
  - L'admin visualizza un messaggio di conferma di salvataggio dell'utente e ritorna alla homepage.
  - L'utente creato, se valido, viene automaticamente salvato nel database e reso disponibile per la gestione futura.

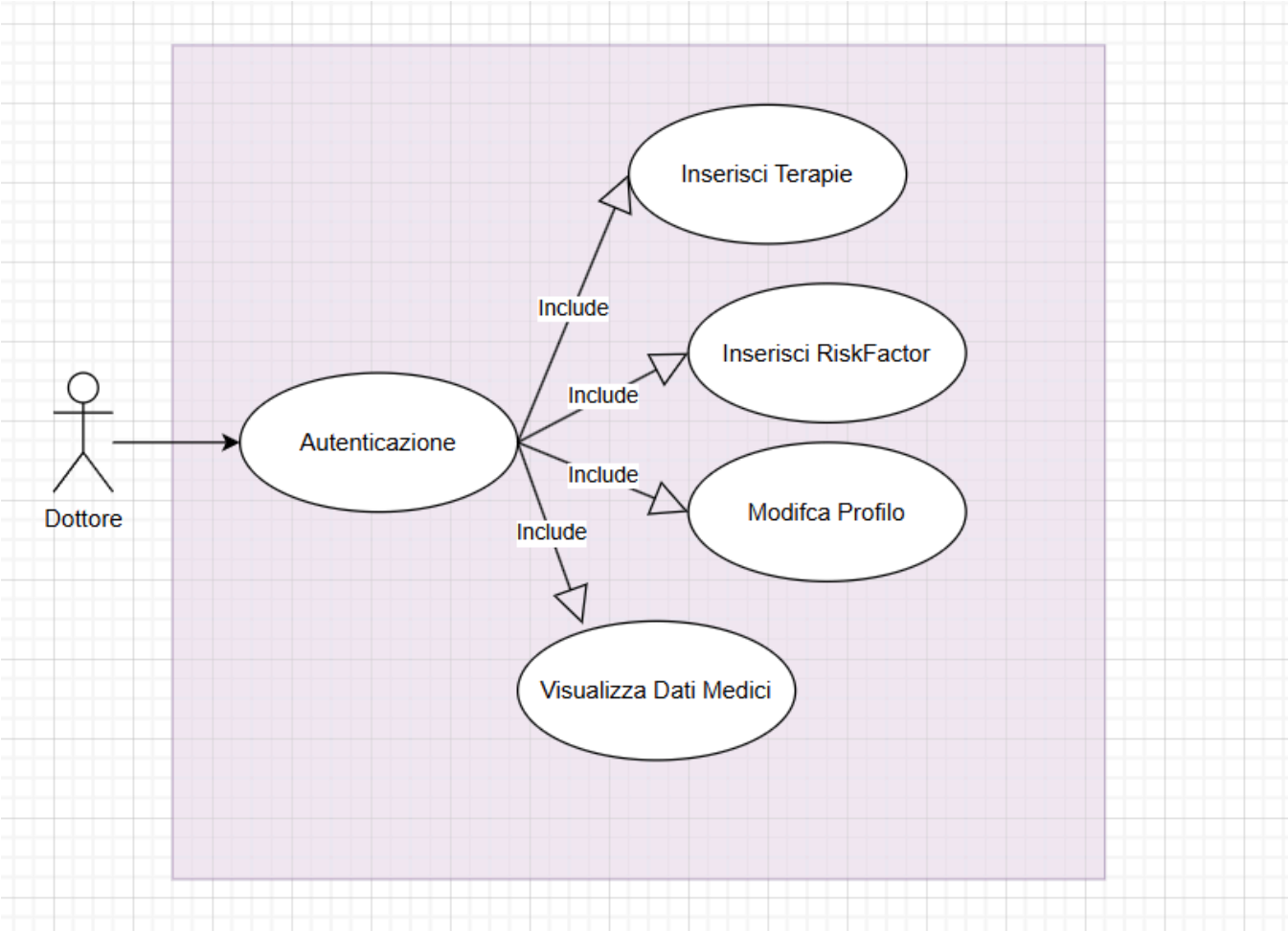
2.1.2 Modifica di un utente già presente

- **Attore:** Admin
- **Precondizioni:** Utente Admin autenticato
- **Passi:**
  1 L'admin accede al sistema
  2 L'admin visualizza gli utenti
  3a. L'admin apre il sottomenu su un utente o apre il profilo di un utente
  3a. L'admin seleziona "Modifica utente"
  4 L'admin modifica i dati dell'utente
  5 Il sistema verifica la validità dei dati
- **Postcondizioni:**
  - L'admin visualizza un messaggio di conferma di salvataggio nell'aggiornamento dei dati e ritorna alla homepage.
  - I dati, se validi, vengono automaticamente aggiornati nel database e resi disponibili.

2.1.3 Cancellazione di un utente già presente

- **Attore:** Admin
- **Precondizioni:** Utente Admin autenticato
- **Passi:**
  1 L'admin accede al sistema
  2 L'admin visualizza gli utenti
  3a L'admin apre il sottomenu dell'utente o apre il profilo di un utente
  3b L'admin seleziona "Cancella utente"
  4 L'admin conferma la cancellazione
  5 Il sistema elimina l'utente dal database
- **Postcondizioni:**
  - L'admin visualizza un messaggio di conferma di cancellazione dell'utente e ritorna alla homepage.
  - L'utenza viene automaticamente eliminata dal database.

2.2 Casi d’uso per il medico



2.2.1 Visualizzazione dei pazienti associati

- **Attore:** Medico
- **Precondizioni:** Medico autenticato
- **Passi:**
  1 Il medico accede al sistema
  2. Il medico seleziona la dashboard pazienti
  3. Il sistema mostra la lista dei pazienti associati al medico
- **Postcondizioni:** Il medico visualizza i dettagli dei pazienti e può selezionarne uno per ulteriori azioni

2.2.2 Consultazione del profilo di un paziente

- **Attore:** Medico

- **Precondizioni:** Medico autenticato
- **Passi:**
  1 Il medico seleziona un paziente dalla lista
  2 Il sistema mostra il profilo dettagliato del paziente (dati anagrafici, misurazioni, sintomi, terapie)
- **Postcondizioni:** Il medico può visualizzare tutte le informazioni cliniche del paziente

2.2.3 Prescrizione o modifica di una terapia

- **Attore:** Medico
- **Precondizioni:** Medico autenticato, paziente selezionato
- **Passi:**
  1 Il medico seleziona la sezione terapie del paziente
  2 Il medico aggiunge o modifica una terapia tramite l'interfaccia dedicata
  3 Il sistema verifica la validità dei dati e aggiorna il database
- **Postcondizioni:** La terapia viene aggiornata e il paziente visualizzerà la modifica nella propria dashboard

> **Nota** Il medico può selezionare il paziente nell'interfaccia di inserimento

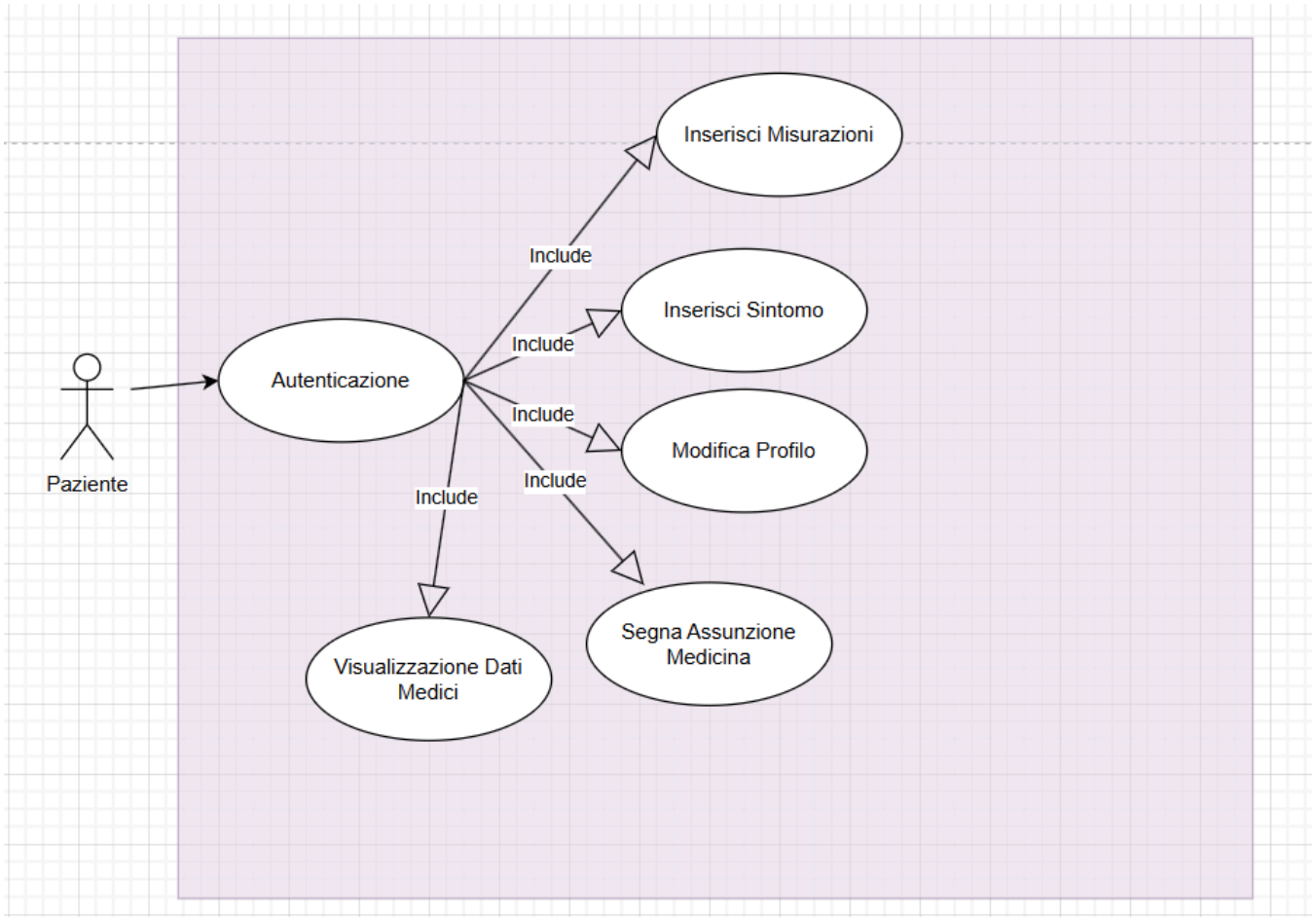
2.2.4 Aggiunta di un risk factor

- **Attore:** Medico
- **Precondizioni:** Medico autenticato, paziente selezionato
- **Passi:**
  1 Il medico seleziona la sezione risk factor del paziente
  2 Il medico aggiunge o modifica un fattore di rischio tramite l'interfaccia dedicata
  3 Il sistema verifica la validità dei dati e aggiorna il database
- **Postcondizioni:** Il fattore di rischio viene aggiornato e si potrà visualizzare

> **Nota** Il medico può selezionare il paziente nell'interfaccia di inserimento

---

2.3 Casi d’uso per il paziente



2.3.1 Inserimento di una misurazione glicemica

- **Attore:** Paziente
- **Precondizioni:** Paziente autenticato
- **Passi:**

  1 Il paziente accede alla dashboard

  1a. Il paziente visualizza le sue misurazioni

  2 Il paziente seleziona “Nuova misurazione”

  3 Il paziente inserisce i dati richiesti e conferma

  4 Il sistema salva la misurazione nel database
- **Postcondizioni:** La misurazione è visibile nello storico e nei grafici personali

2.3.2 Inserimento di un sintomo

- **Attore:** Paziente
- **Precondizioni:** Paziente autenticato
- **Passi:**
  1 Il paziente seleziona “Nuovo sintomo”
  2 Il paziente inserisce la descrizione e conferma
  3 Il sistema salva il sintomo nel database
- **Postcondizioni:** Il sintomo è visibile nello storico e può essere consultato dal medico

2.3.3 Consultazione delle terapie attuali

- **Attore:** Paziente
- **Precondizioni:** Paziente autenticato
- **Passi:**
  1 Il paziente accede alla sezione terapie
  2 Il sistema mostra la lista dei farmaci prescritti e le relative istruzioni
- **Postcondizioni:** Il paziente visualizza le terapie attuali e le eventuali modifiche apportate dal medico

2.3.4 Conferma assunzione delle medicine

- **Attore:** Paziente
- **Precondizioni:** Paziente autenticato, terapie attive
- **Passi**
  1 Il paziente accede alla sezione terapie
  2 Il paziente con il tasto destro apre il sottomenu della medicina da prendere
  3 Il paziente seleziona l'avvenuta assunzione della medicina
  4 Il sistema aggiorna l'assunzione della medicina
- **PostCondizioni:** L'assunzione del medicinale è visibile al paziente.

2.4 Casi d’uso comuni

2.4.1 Modifica del profilo personale

- **Attore:** Qualsiasi utente
- **Precondizioni:** Utente autenticato
- **Passi:**
  1 L’utente accede alla sezione profilo
  2 L’utente modifica i dati e conferma
  3 Il sistema aggiorna il database
- **Postcondizioni:** I dati aggiornati sono visibili nella dashboard

2.4.2 Visualizzazione di alert e notifiche

- **Attore:** Paziente o Medico
- **Precondizioni:** Utente autenticato
- **Passi:**
  1 L’utente accede alla dashboard
  2 Il sistema mostra eventuali alert (valori critici, farmaci in scadenza, sintomi rilevanti)
- **Postcondizioni:** L’utente è informato su situazioni che richiedono attenzione

---

3. Sviluppo

3.1 Stack tecnologico

- **Linguaggio:** Java 17+
- **Framework UI:** JavaFX (FXML, CSS)
- **Database:** SQLite (file locale `glucotrack_db.sqlite`)
- **Build & Dipendenze:** Maven (`pom.xml`)
- **Testing:** JUnit (cartella `test/java`)
- **Gestione risorse:** Struttura organizzata in `assets/` per FXML, CSS, icone e immagini

---

3.2 Gestione progetto

- **Struttura modulare:** Separazione tra model, controller, utilità e view per facilitare la manutenzione e l'estendibilità.
- **Organizzazione delle cartelle:**
  - `src/main/java/it/glucotrack/model/` — Modelli di dominio (es. `Patient`, `Doctor`, `Medication`)
  - `src/main/java/it/glucotrack/controller/` — Controller delle varie dashboard e funzionalità
  - `src/main/java/it/glucotrack/util/` — DAO, gestione database, sessione, mock data
  - `src/main/resources/assets/fxml/` — File FXML per le interfacce grafiche
  - `src/main/resources/assets/css/` — Stili CSS personalizzati
  - `src/main/resources/database/` — Database SQLite e schema SQL
- **Gestione sessione:** Tramite `SessionManager` per il controllo dell'utente autenticato e dei permessi.

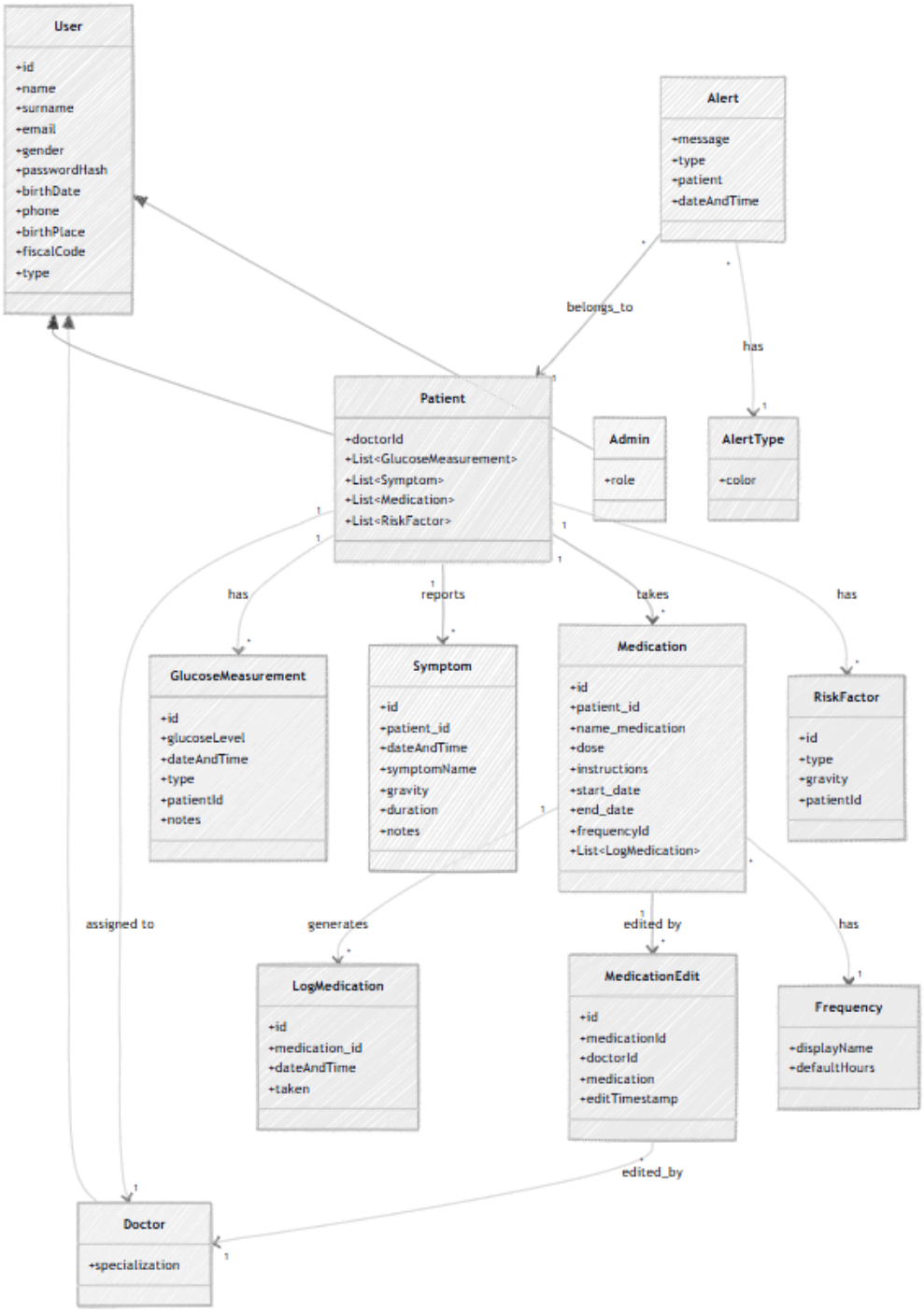
3.3 Librerie

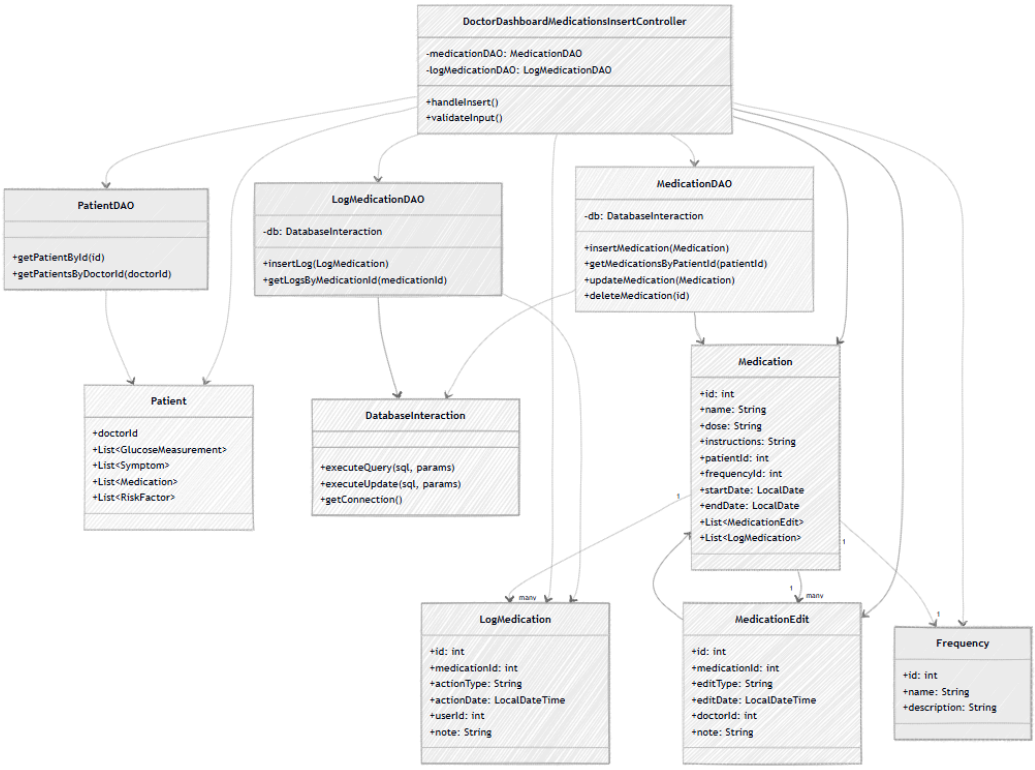
- **JavaFX:** Per la realizzazione dell'interfaccia grafica e la gestione degli eventi.
- **SQLite JDBC:** Per la persistenza dei dati.
- **JUnit:** Per i test automatici.
- **Maven:** Per la gestione delle dipendenze e la build del progetto.

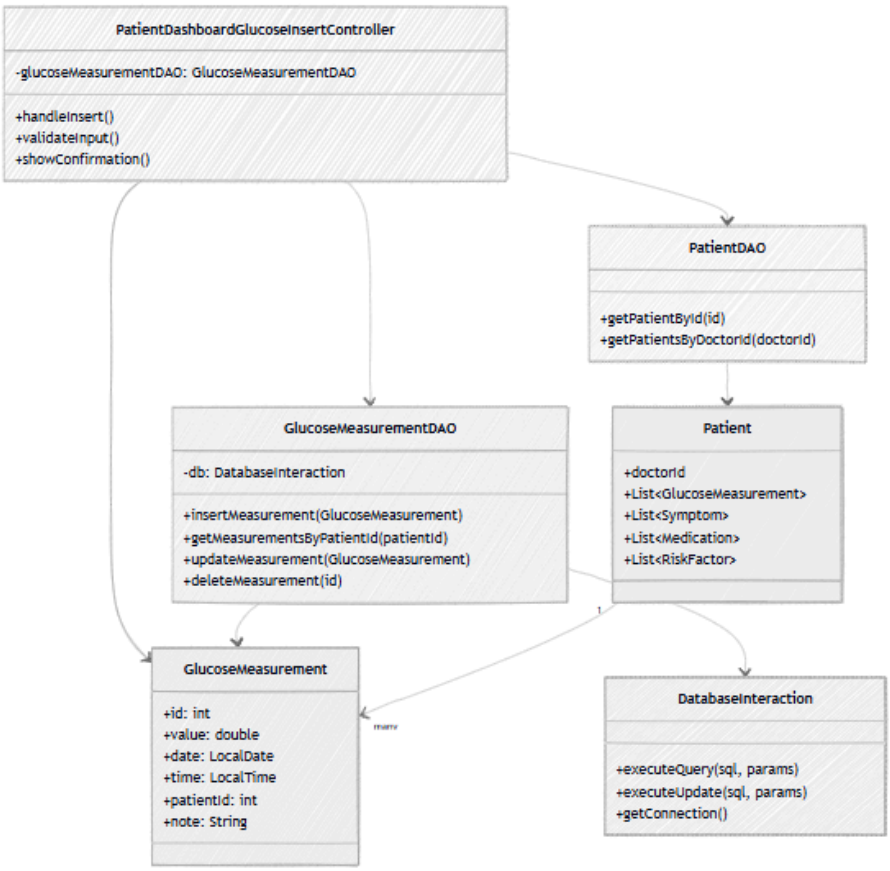
3.4 Design Pattern

- **MVC (Model-View-Controller):** Separazione tra dati, logica e interfaccia.
- **DAO (Data Access Object):** Incapsulamento dell'accesso ai dati tramite oggetti dedicati.
- **Singleton:** Gestione della sessione utente (`SessionManager`).
- **Observer:** Proprietà osservabili per aggiornare la UI in tempo reale.
- **Factory:** Creazione dinamica di view e oggetti model.
- **Template Method:** Struttura comune per le operazioni sui dati.

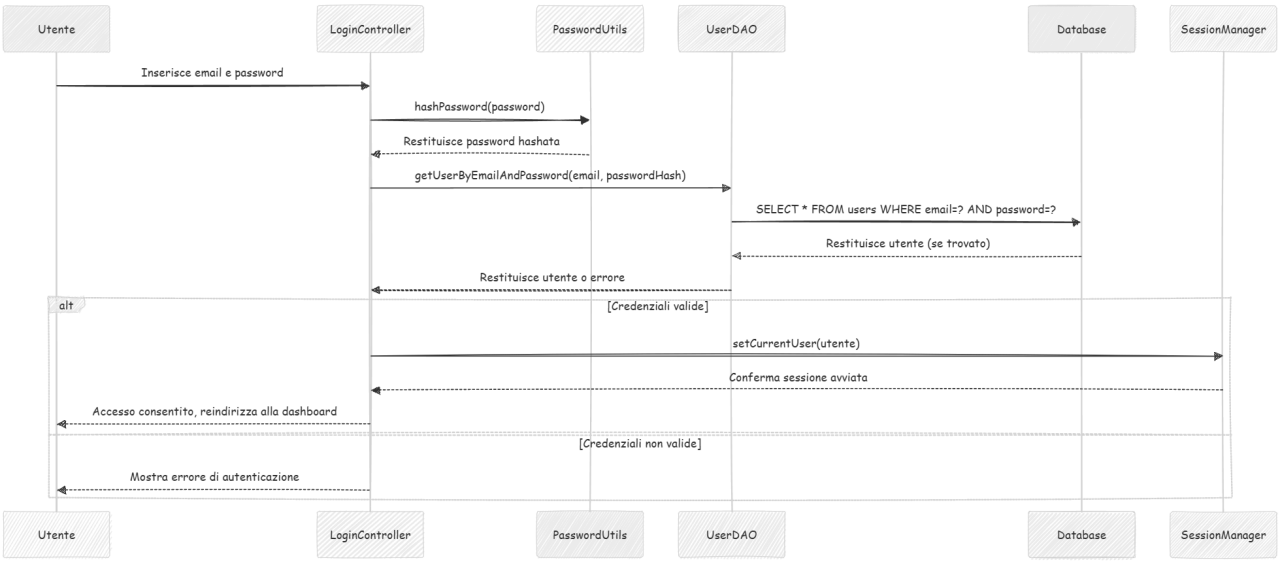
3.5.1 Diagrammi delle classi

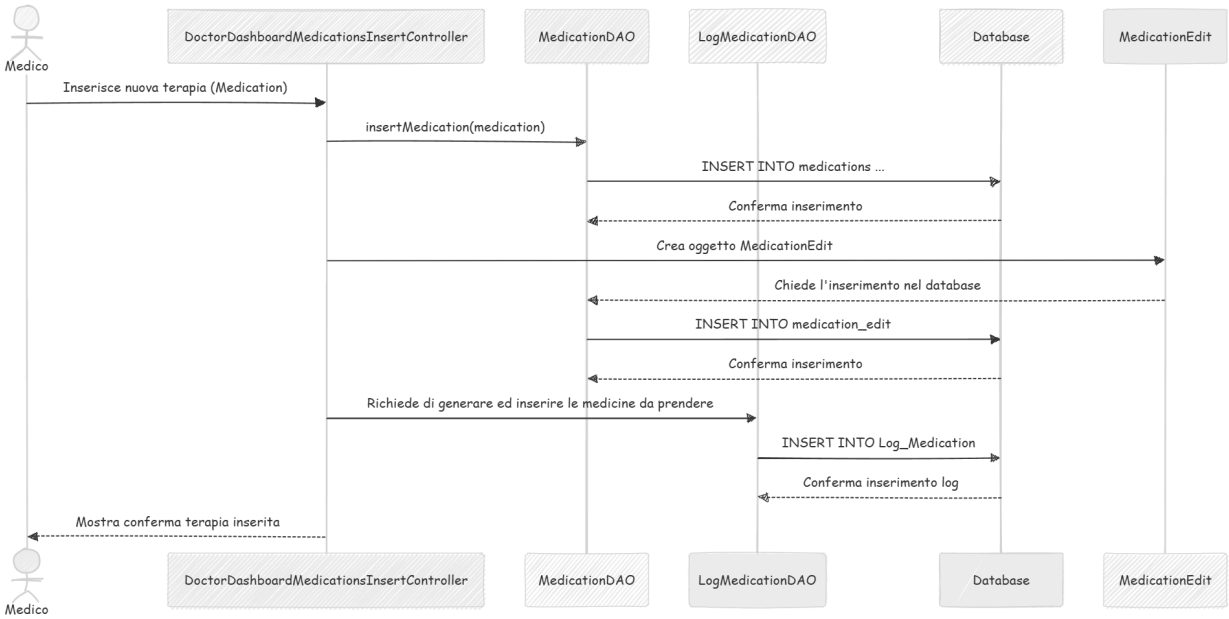


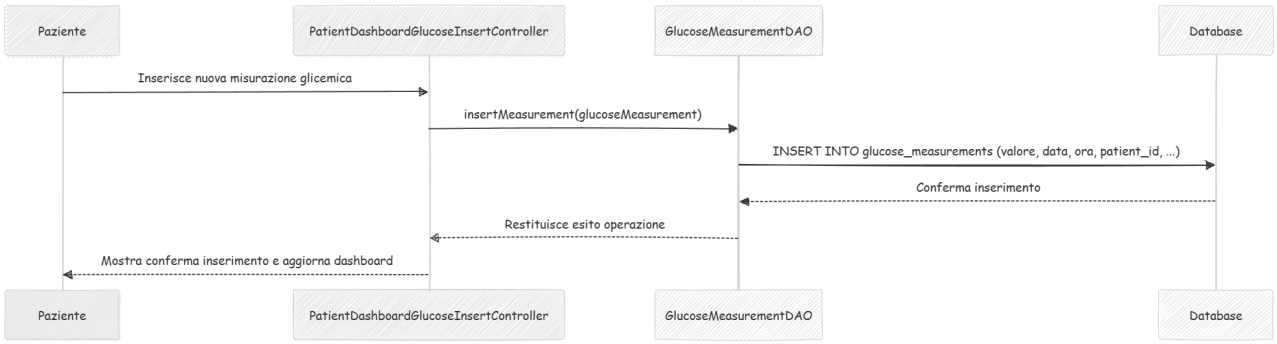

DoctorDashboardMedicationsInsertController
-medicationDAO: MedicationDAO
-logMedicationDAO: LogMedicationDAO
+handleInsert()
+validateInput()
PatientDAO
+getPatientById(id)
+getPatientsByDoctorId(doctorId)
LogMedicationDAO
-db: DatabaseInteraction
+insertLog(LogMedication)
+getLogsByMedicationId(medicationId)
MedicationDAO
-db: DatabaseInteraction
+insertMedication(Medication)
+getMedicationsByPatientId(patientId)
+updateMedication(Medication)
+deleteMedication(id)
Patient
+doctorId
+List<GlucoseMeasurement>
+List<Symptom>
+List<Medication>
+List<RiskFactor>
DatabaseInteraction
+executeQuery(sql, params)
+executeUpdate(sql, params)
+getConnection()
Medication
+id: int
+name: String
+dose: String
+instructions: String
+patientId: int
+frequencyId: int
+startDate: LocalDate
+endDate: LocalDate
+List<MedicationEdit>
+List<LogMedication>
1
many
1
many
1
1
LogMedication
+id: int
+medicationId: int
+actionType: String
+actionDate: LocalDateTime
+userId: int
+note: String
MedicationEdit
+id: int
+medicationId: int
+editType: String
+editDate: LocalDateTime
+doctorId: int
+note: String
Frequency
+id: int
+name: String
+description: String

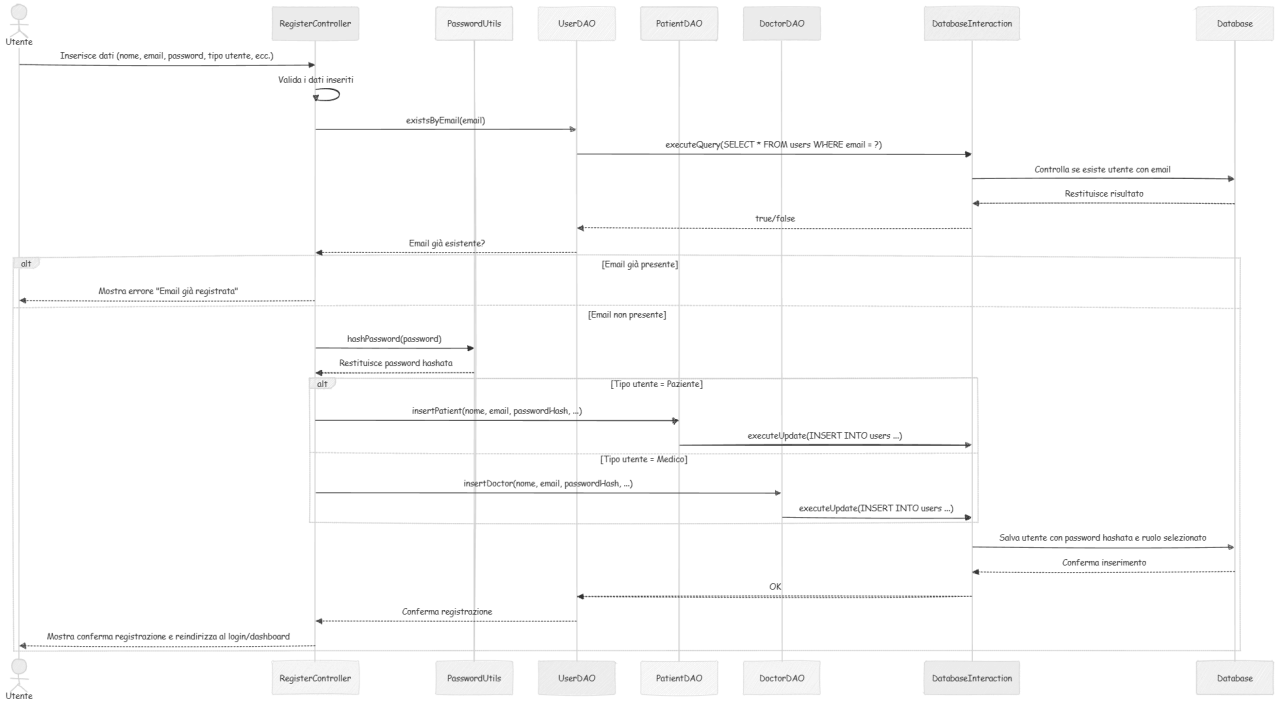

PatientDashboardGlucoseInsertController
-glucoseMeasurementDAO: GlucoseMeasurementDAO
+handleInsert()
+validateInput()
+showConfirmation()
PatientDAO
+getPatientById(id)
+getPatientsByDoctorId(doctorId)
GlucoseMeasurementDAO
-db: DatabaseInteraction
+insertMeasurement(GlucoseMeasurement)
+getMeasurementsByPatientId(patientId)
+updateMeasurement(GlucoseMeasurement)
+deleteMeasurement(id)
Patient
+doctorId
+List<GlucoseMeasurement>
+List<Symptom>
+List<Medication>
+List<RiskFactor>
1
GlucoseMeasurement
+id: int
+value: double
+date: LocalDate
+time: LocalTime
+patientId: int
+note: String
many
DatabaseInteraction
+executeQuery(sql, params)
+executeUpdate(sql, params)
+getConnection()

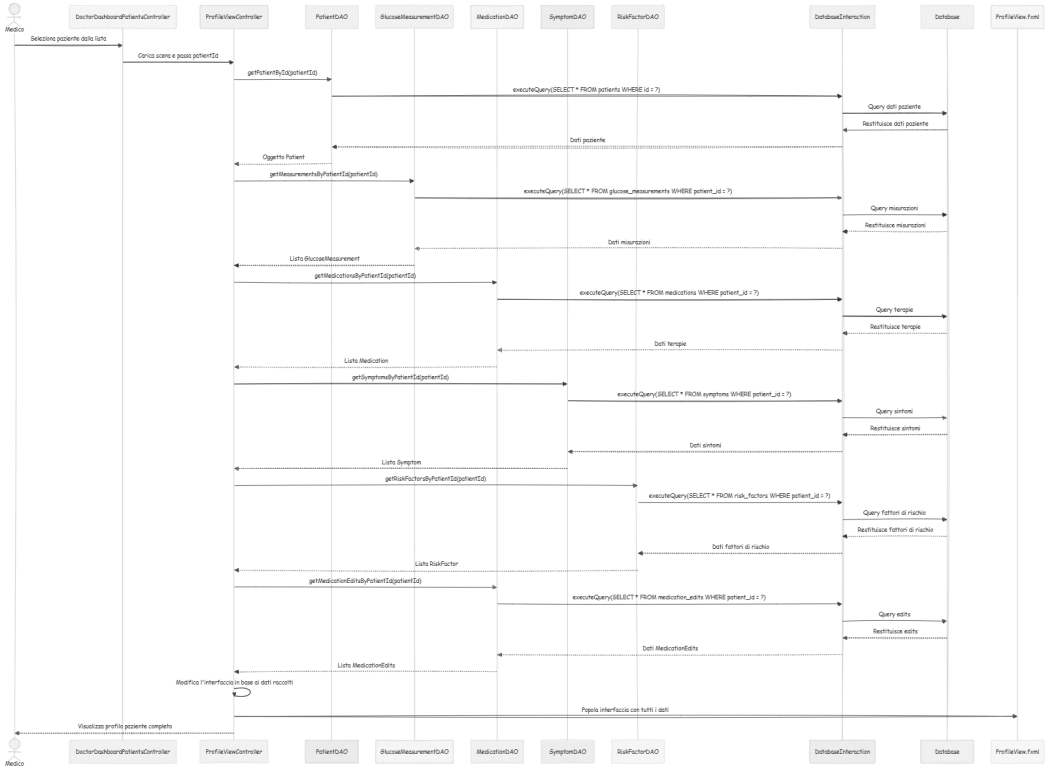
3.5.2 Diagrammi di sequenza



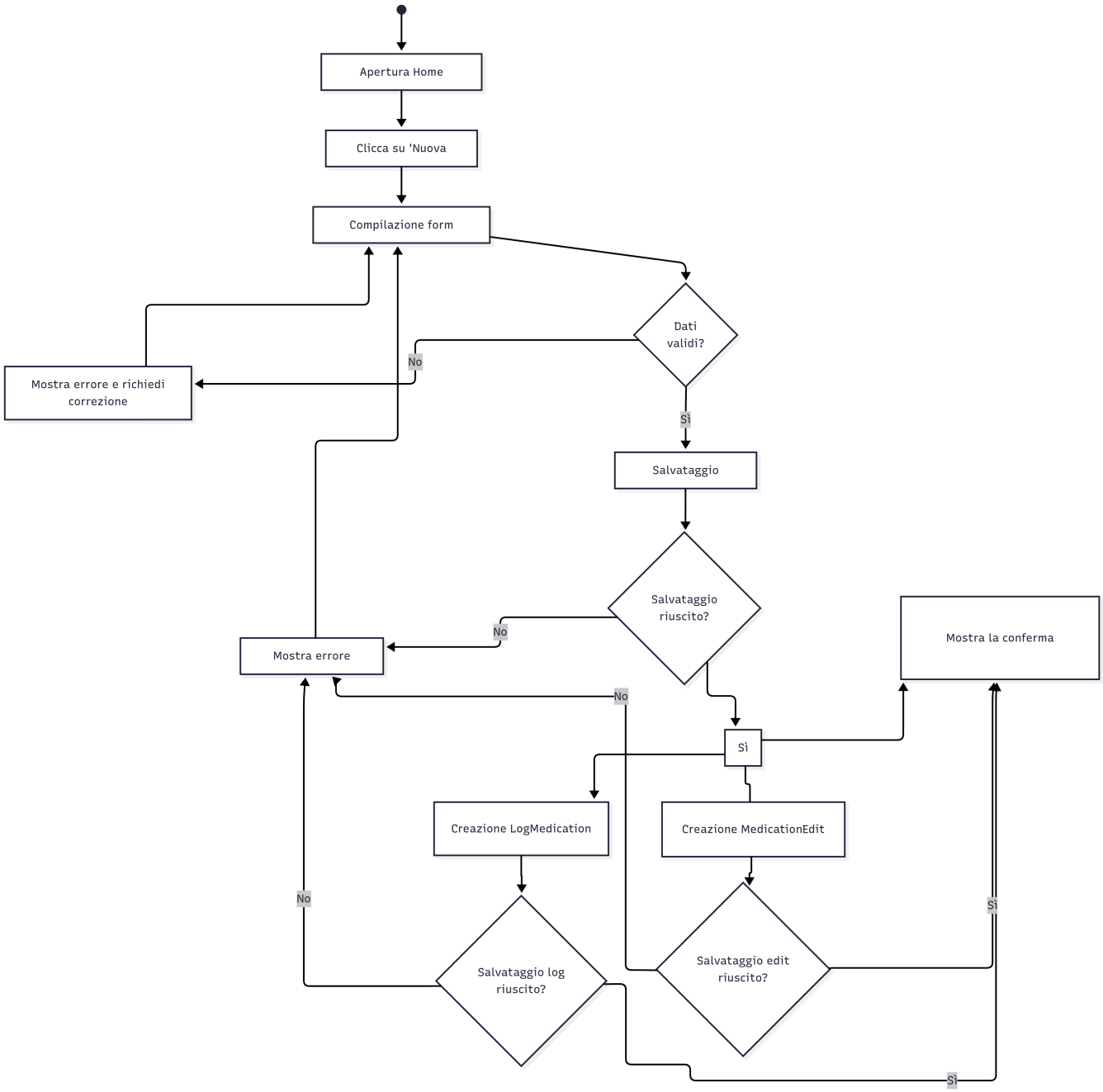


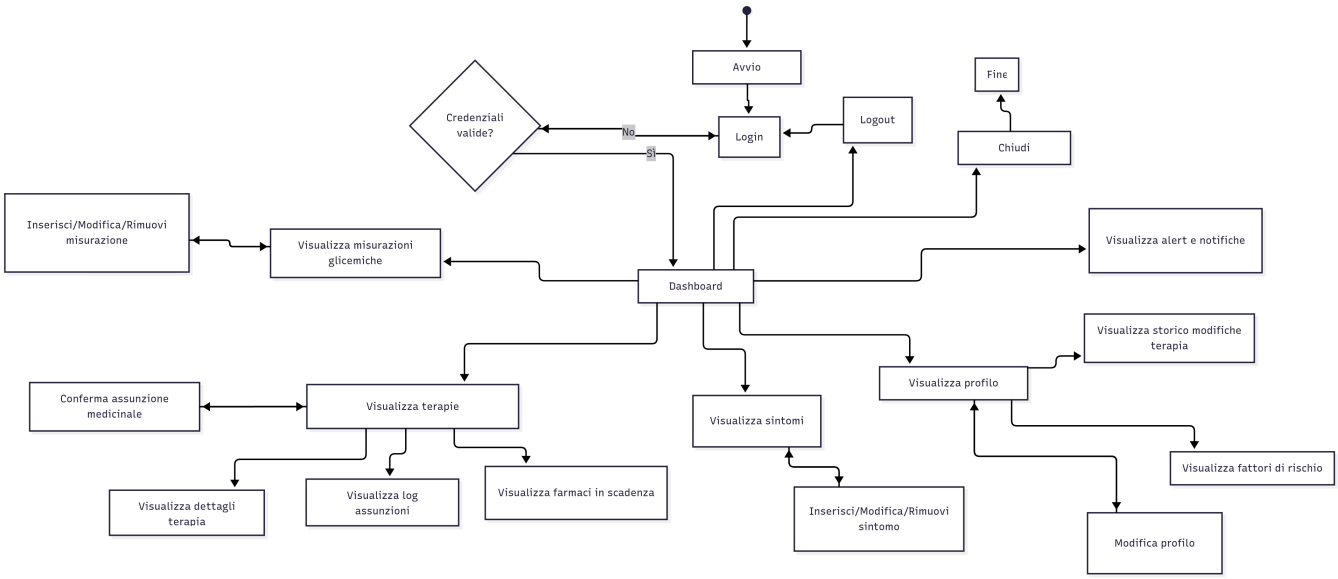

Paziente
PatientDashboardGlucoseInsertController
GlucoseMeasurementDAO
Database
Inserisce nuova misurazione glicemica
insertMeasurement(glucoseMeasurement)
INSERT INTO glucose_measurements (valore, data, ora, patient_id, ...)
Conferma inserimento
Restituisce esito operazione
Mostra conferma inserimento e aggiorna dashboard
Paziente
PatientDashboardGlucoseInsertController
GlucoseMeasurementDAO
Database


Utente
RegisterController
PasswordUtils
UserDAO
PatientDAO
DoctorDAO
DatabaseInteraction
Database
Inserisce dati (nome, email, password, tipo utente, ecc.)
Valida i dati inseriti
existsByEmail(email)
executeQuery(SELECT * FROM users WHERE email = ?)
Controlla se esiste utente con email
Restituisce risultato
true/false
Email già esistente?
alt
[Email già presente]
Mostra errore "Email già registrata"
[Email non presente]
hashPassword(password)
Restituisce password hashata
alt
[Tipo utente = Paziente]
insertPatient(nome, email, passwordHash, ...)
executeUpdate(INSERT INTO users ...)
[Tipo utente = Medico]
insertDoctor(nome, email, passwordHash, ...)
executeUpdate(INSERT INTO users ...)
Salva utente con password hashata e ruolo selezionato
Conferma inserimento
OK
Conferma registrazione
Mostra conferma registrazione e reindirizza al login/dashboard
Utente
RegisterController
PasswordUtils
UserDAO
PatientDAO
DoctorDAO
DatabaseInteraction
Database


Medico
DoctorDashboardPatientsController
ProfileViewController
PatientDAO
GlucoseMeasurementDAO
MedicationDAO
SymptomDAO
RiskFactorDAO
DatabaseInteraction
Database
ProfileView.fxml
Seleziona paziente dalla lista
Carica scena e passa patientId
getPatientById(patientId)
executeQuery(SELECT * FROM patients WHERE id = ?)
Query dati paziente
Restituisce dati paziente
Dati paziente
Oggetto Patient
getMeasurementsByPatientId(patientId)
executeQuery(SELECT * FROM glucose_measurements WHERE patient_id = ?)
Query misurazioni
Restituisce misurazioni
Dati misurazioni
Lista GlucoseMeasurement
getMedicationsByPatientId(patientId)
executeQuery(SELECT * FROM medications WHERE patient_id = ?)
Query terapie
Restituisce terapie
Dati terapie
Lista Medication
getSymptomsByPatientId(patientId)
executeQuery(SELECT * FROM symptoms WHERE patient_id = ?)
Query sintomi
Restituisce sintomi
Dati sintomi
Lista Symptom
getRiskFactorsByPatientId(patientId)
executeQuery(SELECT * FROM risk_factors WHERE patient_id = ?)
Query fattori di rischio
Restituisce fattori di rischio
Dati fattori di rischio
Lista RiskFactor
getMedicationEditsByPatientId(patientId)
executeQuery(SELECT * FROM medication_edits WHERE patient_id = ?)
Query edits
Restituisce edits
Dati MedicationEdits
Lista MedicationEdits
Modifica l'interfaccia in base ai dati raccolti
Popola interfaccia con tutti i dati
Visualizza profilo paziente completo

3.5.3 Diagrammi di stato





---

4 Validazione e Test

4.1 MockData

Per facilitare la fase di test e validazione del sistema, è stata implementata la classe `DatabaseMockData` all'interno della cartella `util/`.
Questa classe consente di **popolare automaticamente il database** con utenti di esempio (admin, medici, pazienti) e dati clinici (misurazioni, sintomi, terapie), permettendo di simulare scenari realistici e di verificare il corretto funzionamento delle funzionalità principali dell'applicazione.

Durante lo sviluppo, la classe mock è stata utilizzata per:

- **Generare utenti di test** con dati validi e diversificati, utili per verificare la gestione delle autenticazioni, dei permessi e delle dashboard dedicate.
- **Popolare le tabelle chiave** del database (`users`, `patients`, `doctors`, ecc.) senza dover inserire manualmente i dati tramite interfaccia.
- **Testare i flussi di login, registrazione, visualizzazione e modifica** degli utenti, assicurando che le logiche di business e di sicurezza (come la crittografia delle password)

funzionassero correttamente anche con dati generati automaticamente.

L'utilizzo della classe mock ha permesso di:

- **Effettuare test di integrazione** su tutte le funzionalità utente-sistema.
- **Verificare la robustezza delle query e dei DAO** in presenza di dati eterogenei.
- **Accelerare la fase di debug** e la validazione delle interfacce grafiche, grazie alla disponibilità immediata di dati di esempio.

> **Nota:** Per garantire la sicurezza, è stato necessario assicurarsi che anche le password generate dalla classe mock fossero correttamente hashate prima dell'inserimento nel database, così da simulare fedelmente il comportamento in produzione.

4.2 Unit Test

Durante la fase di test, il programma GlucoTrack è stato sottoposto a prove pratiche da parte di utenti di diversa esperienza: studenti nel settore sanitario, studenti di informatica e utenti meno esperti.

I risultati emersi sono stati i seguenti:

- **Studenti del settore sanitario** hanno apprezzato la chiarezza delle dashboard e la semplicità nell'inserimento di misurazioni, sintomi e terapie. Hanno trovato utile la visualizzazione dello storico e delle statistiche, segnalando che la struttura delle informazioni rispecchia le esigenze cliniche reali.
- **Studenti di informatica** hanno testato le funzionalità più tecniche, come la gestione degli utenti, la sicurezza delle password e la robustezza delle query. Hanno confermato che la crittografia delle password funziona correttamente e che i dati mock permettono di simulare scenari realistici. Alcuni hanno suggerito miglioramenti nell'usabilità delle interfacce e nella gestione degli errori.
- **Utenti meno esperti** hanno riscontrato una buona intuitività generale, riuscendo a completare le operazioni principali (login, inserimento dati, consultazione delle terapie) senza difficoltà. Tuttavia, hanno suggerito di aggiungere ulteriori messaggi di aiuto e conferma, soprattutto nelle fasi di registrazione e modifica del profilo.

Nel complesso, i test hanno evidenziato che GlucoTrack è accessibile e funzionale per tutte le categorie di utenti, con una curva di apprendimento rapida e una gestione sicura dei dati sensibili. I feedback raccolti sono stati utilizzati per ottimizzare la user experience e migliorare

la documentazione di supporto.

4.3 Tabella generale: Lati positivi e negativi emersi dai test

Lati Positivi	Lati Negativi
Interfaccia intuitiva e facile da usare per tutte le categorie di utenti	Alcuni utenti meno esperti hanno richiesto più messaggi di aiuto e conferma
Dashboard chiare e ben organizzate	Alcuni studenti di informatica hanno suggerito miglioramenti nell’usabilità
Inserimento e consultazione di misurazioni, sintomi e terapie semplice e veloce	Gestione degli errori migliorabile in alcune sezioni
Visualizzazione dello storico e delle statistiche utile per il settore sanitario	Alcune funzionalità avanzate richiedono una guida più dettagliata
Sicurezza dei dati garantita (crittografia password, gestione permessi)	Documentazione di supporto da ampliare secondo i feedback ricevuti